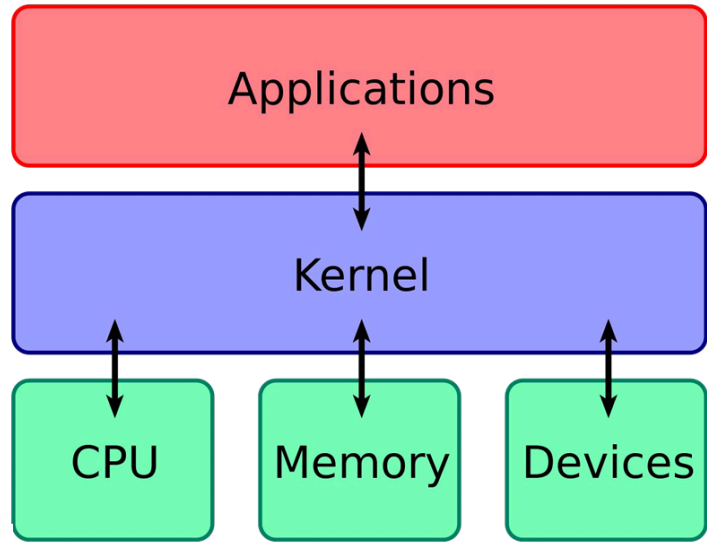




Ointro

Kernel: Sistem kaynaklarını kontrol edip uygulamalarla ilişkilerini sağlayan ara katmandır.



Bir Programın Derlenmesi

1. Doğrudan Derlenen Diller

- C ve C++ gibi dillerin kaynak kodu direkt olarak binary koda dönüştürülebilir.
- Bu diller daha hızlı çalışır.
- Ancak bu durum OS'e bağımlı olmalarına sebep olur ve Windows'ta çalışırken Linux'ta çalışamazlar.

2. Sanal Makine Üzerinde Çalışan Diller

- Java gibi diller direkt makine koduna dönüştürülmek yerine bytecode adı verilen özel ve sadece JVM (Java Virtual Machine) tarafından anlaşılacak bir dile çevirir.

- b. OS, java kodunu değil JVM'yi çalıştırır. Bu yüzden java "bir kez yaz, istediğin yerde çalıştır" prensibine sahiptir.

3. Yorumlanan Diller

- a. Python, kodu satır satır okuyup anında makine diline çevirip çalıştırır.
- b. Tıpkı javada olduğu gibi OS python kodunu değil, python yorumlayıcısını (interpreter) çalıştırır.

Process (İşlem)

- **Program:**
 - Sabit diskteki binary koddur.
 - Onu çalıştırmak için verilerini ve talimatlarını belleğe yükleyip CPU'da çalıştırmamız gerekir.
- **Process:**
 - Belleğe yüklenen programdır.

Multi-Tasking

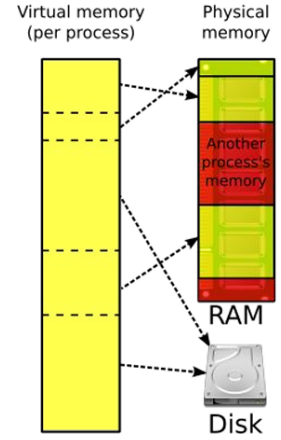
- Bazı programlar aynı belleği, işlemciyi ve i/o cihazlarını kullanmak isteyebilir.
- Bu durumda bazı problemler oluşur:
 - Bir program diğerini etkileyebilir.
 - Bir program diğer programın verilerini çalabilir.
 - Birden fazla kullanıcı olabilir.

Protection

- Process'lerin izole edilmesi:
 - İşlemlerin kullandıkları bellekler birbirinden izole edilir.

Sistem Kaynaklarının Sanallaştırılması

- OS her process için sanal ve tek parça bir bellek alanı oluşturur.
- Aslında bu alan gerçek bellek ve sabit diskin rastgele alanlarının birleştirilmesiyle oluşturulmuştur.
- Ancak process bunu bilmez ve tamamı ona ait olan bir belleği olduğunu zanneder.

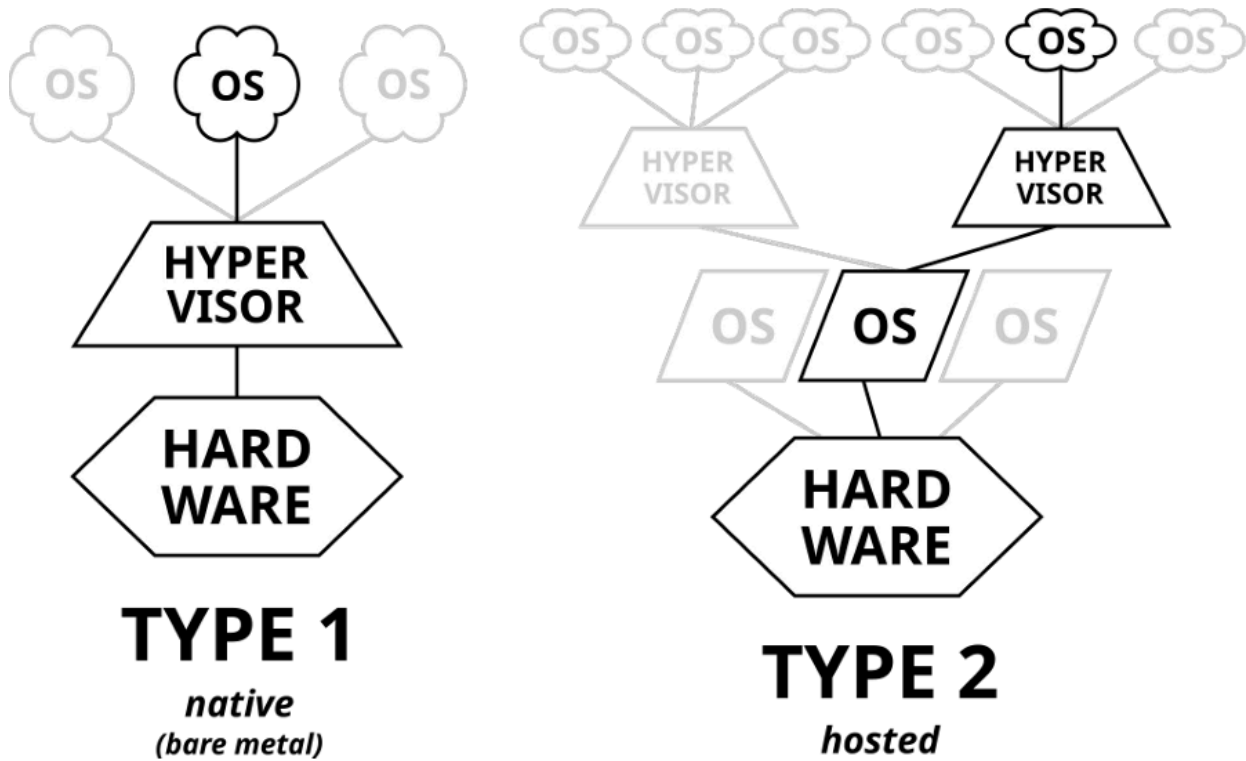


Sistem Seviyesinde Sanallaştırma

Hypervisor

- VirtualBox gibi virtual machine'ler örnek verilebilir.
- OS uygulamalara ve son kullanıcıya hizmet ederken hypervisor'lar sadece OS'lere hizmet ederler.

Sistem Sanallaştırma Mimarileri



1. Type 1 (Native)
 - a. Veri merkezlerinde ve sunucularda kullanılır.
 - b. Type 2'ye göre daha hızlıdır.
2. Type 2 (Hosted)
 - a. Kişisel bilgisayarlarda kullanılır.
 - b. En üstte yapılan işlem sırayısla OS (exp. ubuntu 22.04) > Hypervisor (exp. VirtualBox) > OS (exp. windows 11) > Hardware yolunu izlediği için daha verimsizdir.

QEMU (Quick Emulator)

- QEMU bir emülatör ve sanallaştırıcıdır.

1. Sanallaştırma

- a. Misafir işletim sistemiyle bilgisayar donanımındaki işlemcinin aynı mimariye sahip olduğu durumdur.

- i. Örneğin; Intel (x86) işlemcili bir Windows bilgisayarda, yine Intel (x86) için yazılmış bir Ubuntu Linux çalıştırıyorsun. İşlemci komutları aynı olduğu için, Hypervisor bu komutları doğrudan donanıma iletir.
- b. Çok hızlıdır çünkü aracı (çevirmen) yoktur. İşlemci komutları doğrudan anlar.

2. Emülasyon

- a. Misafir işletim sisteminin, senin asıl bilgisayarından **tamamen farklı bir işlemci mimarisine** veya donanıma sahip olduğu durumdur.
 - i. Örneğin; bilgisayarında eski bir Gameboy oyununu oynamak veya Intel (x86) işlemcili bilgisayarında telefonlar için yazılmış ARM mimarili bir Android işletim sistemini çalıştırmak istiyorsun. İşlemcin bu komutları doğrudan anlayamaz!
- b. QEMU yazılımsal bir "çevirmen" gibi çalışır. ARM komutlarını alır, anında Intel'in anlayacağı komutlara çevirir ve donanıma öyle yollar. Ayrıca sadece işlemciyi değil; ekran kartını, ağ kartını, USB portlarını bile yazılımsal olarak taklit eder (emulate eder).
- c. Her komut anında çevrildiği için sanallaştırmaya göre **çok daha yavaştır**, ancak sana normalde o cihazda çalışmayacak bir sistemi çalıştırma özgürlüğü verir.

Containerization

